CSE115 – Section 5 | Project Group No: 6
Mehrab Hasan (ID: 2613095042)
Wasi Al Imran (Id: 2613115042)
Tahmeed Akanda Protik (Id: 2611639042)
Azman Sami (Id: 2611617042)

Pixel Art Editor

Abstract

This project is a Pixel Art Editor that is made using the C programming language. It has a canvas that's like a grid with 20 rows and 20 columns. On this canvas people can make pixel art by drawing and erasing pixels. They can also fill pixels with colors. The Pixel Art Editor uses numbers to pick the colors for the pixels.

The main reason for making the Pixel Art Editor is to show how to use some things in the C programming language. These things are. Functions and loops and conditional statements. The project also shows how to make a menu that the user can interact with. The Pixel Art Editor gets input from the user through the console. Then it updates the canvas to show what the user has done.

The Pixel Art Editor is a tool, for beginners who want to make digital drawings. It. Signs to show blank pixels. It uses numbers to show pixels that have color. The Pixel Art Editor is a way for people to make pixel art on their computer.

Introduction

The goal of this project is to create a text-based Pixel Art Editor that people can use. This program lets users play around with a grid by adding pixels removing selected pixels filling the whole grid with a color they like and showing the updated picture.

Pixel art is when you make pictures using blocks or pixels. This project does something but in a simple text-based way. Of using actual pixels the program uses a grid to show the picture. Each little square in the grid is like one pixel. If the square is zero it means the pixel is blank. If it is any number it means the pixel is colored.

This project is important because it teaches programming ideas like using variables that are available everywhere loops inside loops getting input from users checking boundaries making functions using arrays and making menus with choices. These ideas are necessary for making programs in C. We also need to write a report about this project, which will have parts, like a summary, introduction, what it can do and how it works. This project fits into all those parts.

CSE115 – Section 5 | Project Group No: 6
Mehrab Hasan (ID: 2613095042)
Wasi Al Imran (Id: 2613115042)
Tahmeed Akanda Protik (Id: 2611639042)
Azman Sami (Id: 2611617042)

Features and Implementation

Feature 1: Grid Initialization

The program uses a global 20×20 integer array named grid as the drawing canvas. The InitializeGrid() function sets every cell of the grid to 0, meaning all pixels are initially blank.

```
int grid[20][20];
int size = 20;
```

The initialization is done using nested for loops. Each row and column position is assigned the value 0.

```
void InitializeGrid(){
   for (int i = 0; i < size; i++){
     for (int j = 0; j < size; j++){
      grid[i][j]=0;
      }
   }
}
```

This ensures that the canvas is empty when the program starts.

Feature 2: Display Grid

The DisplayGrid() function shows the current state of the canvas. It checks each cell of the grid. If the value is 0, it prints + to represent a blank pixel. Otherwise, it prints the stored color number.

```
if (grid[i][j]==0){
  printf(" + ");
}
else{
  printf(" %d ",grid[i][j]);
}
```

This allows the user to visually track changes after drawing, erasing, or filling the grid.

CSE115 – Section 5 | Project Group No: 6
Mehrab Hasan (ID: 2613095042)
Wasi Al Imran (Id: 2613115042)
Tahmeed Akanda Protik (Id: 2611639042)
Azman Sami (Id: 2611617042)

Feature 3: Draw Pixel

The DrawPixel() function allows the user to enter a row, column, and color number. Since users enter positions starting from 1, the program subtracts 1 from the row and column values to match C's zero-based array indexing.

```
row--;
col--;
```

The program then checks whether the selected position is inside the valid grid range. If valid, the selected cell is updated with the color number.

```
if(row >= 0 && row < size && col >= 0 && col < size) {
   grid[row][col] = color;
}
```

This prevents the user from accessing invalid array positions.

Feature 4: Erase Pixel

The ErasePixel() function works similarly to the drawing function. The user enters a row and column, and the program sets that grid cell back to 0.

```
grid[row][col] = 0;
```

This removes the color from that pixel and makes it blank again. Boundary checking is also used here to prevent invalid positions.

Feature 5: Fill Grid

The FillGrid() function allows the user to fill the entire canvas with a single color number. It takes a color input from the user and assigns that value to every cell in the grid using nested loops.

```
for (int i = 0; i < size; i++)
{
  for (int j = 0; j < size; j++){
     grid[i][j]=color;
  }
}
```

This feature is useful when the user wants to quickly set the whole canvas to one color.

Feature 6: Menu-Driven System

CSE115 – Section 5 | Project Group No: 6
Mehrab Hasan (ID: 2613095042)
Wasi Al Imran (Id: 2613115042)
Tahmeed Akanda Protik (Id: 2611639042)
Azman Sami (Id: 2611617042)

The project uses a menu inside the main() function. The menu continues running until the user chooses option 5 to exit.

```
while (choice!=5)
```

A switch statement is used to call the correct function based on the user's choice.

```
switch (choice){
   case 1: DisplayGrid(); break;
   case 2: DrawPixel(); DisplayGrid(); break;
   case 3: ErasePixel(); DisplayGrid(); break;
   case 4: FillGrid(); DisplayGrid(); break;
   case 5: return 0;
   default: printf("!!!!Invalid Choice.Try Again!!!!\n");
}
```

This makes the program interactive and easy to use. After drawing, erasing, or filling, the updated grid is displayed automatically.

Member Contribution

Feature/Task	Implemented By	Description
Project Planning & Guidance	Md Wasi Al Imran(2613115042)	Led the project by explaining the overall concept, structure, and workflow to the team. Helped others understand how the pixel art editor should function and coordinated the implementation approach.
Drawing Tool	Azman Sami(2611617042)	Developed the functionality to draw individual pixels by taking user input (row, column, and color) and updating the grid accordingly with proper boundary checking.
Fill Color Tool	Tahmeed Akanda Protik(2611639042)	Implemented the feature to fill the entire grid with a single color using nested loops, allowing quick modification of the whole canvas.
Erase Tool	Mehrab Hasan(2613095042)	Created the erase functionality that resets selected pixels back to blank (value 0), ensuring proper validation of user input positions.

CSE115 – Section 5 | Project Group No: 6
Mehrab Hasan (ID: 2613095042)
Wasi Al Imran (Id: 2613115042)
Tahmeed Akanda Protik (Id: 2611639042)
Azman Sami (Id: 2611617042)

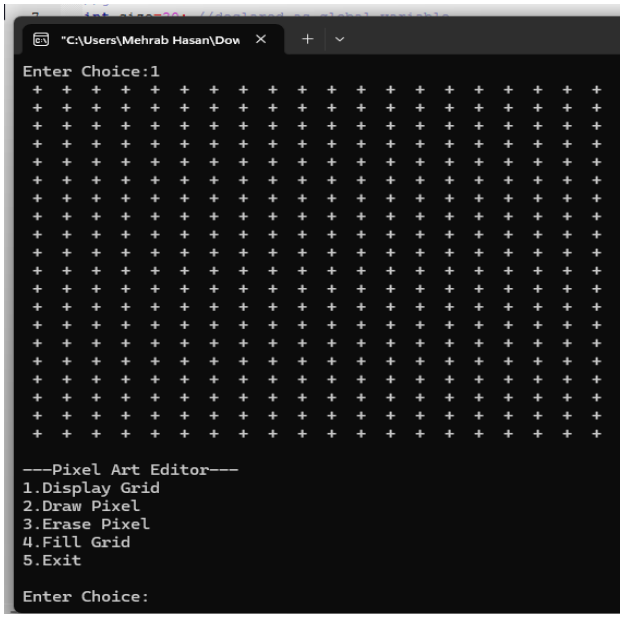
Testing and Results

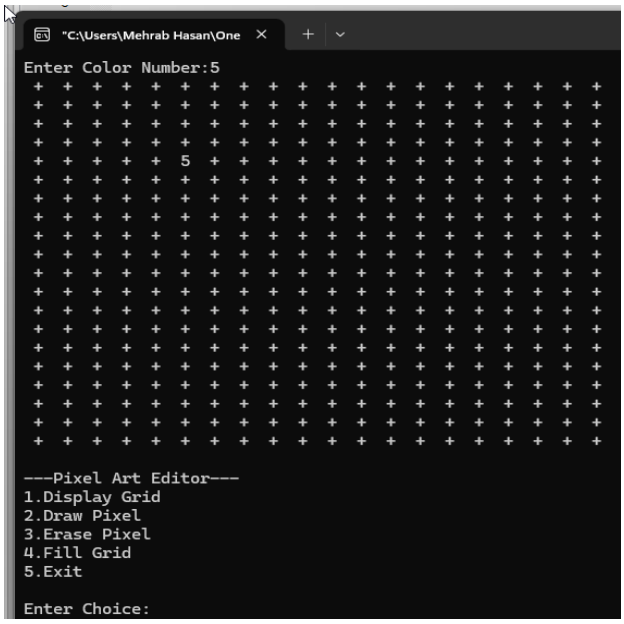
Test cases:

Input Scenario	Expected Output	Result
Display Grid (initial state)	Empty grid shown using + symbols	Passed
Draw Pixel (valid position)	Selected pixel updated with given color number	Passed
Draw Pixel (invalid position)	Error message: "Invalid position"	Passed
Erase Pixel (valid position)	Selected pixel reset to + (blank)	Passed
Erase Pixel (invalid position)	Error message: "Invalid position"	Passed
Fill Grid	Entire grid filled with the entered color	Passed
Menu Choice (invalid input)	Error message: "Invalid Choice. Try Again"	Passed
Exit Option	Program terminates successfully	Passed

CSE115 – Section 5 | Project Group No: 6
Mehrab Hasan (ID: 2613095042)
Wasi Al Imran (Id: 2613115042)
Tahmeed Akanda Protik (Id: 2611639042)
Azman Sami (Id: 2611617042)

Screenshots:





Results:

All features of the Pixel Art Editor were tested with different input scenarios. The program handled both valid and invalid inputs correctly. Boundary checks worked as expected, and all operations (draw, erase, fill, display) performed reliably without crashes. Overall, the system passed all test cases successfully.

Challenges and Solutions

Challenges Faced:

- Taking and processing user inputs (row, column, color) without errors.
- Dealing with the difference between user-friendly numbering (starting from 1) and C array indexing (starting from 0).
- Making sure users cannot access positions outside the 20×20 grid.
- Presenting the grid clearly in a text-based console so it is easy to understand.
- Keeping the menu system running smoothly without breaking the program flow.

Solutions:

- Applied proper input validation using conditional checks before updating the grid.
- Converted user inputs to correct array indices by subtracting 1 from row and column values.
- Added boundary checking conditions to prevent invalid grid access.
- Used + symbols for empty pixels and numbers for colored pixels to improve readability.
- Implemented a loop-based menu with switch-case statements to ensure continuous and stable execution.

CSE115 – Section 5 | Project Group No: 6
Mehrab Hasan (ID: 2613095042)
Wasi Al Imran (Id: 2613115042)
Tahmeed Akanda Protik (Id: 2611639042)
Azman Sami (Id: 2611617042)

Conclusion

Summary of the Project:

The **Pixel Art Editor** project successfully demonstrates the implementation of a simple drawing system using C programming. It includes core features such as drawing pixels, erasing pixels, filling the grid, and displaying the canvas through a user-friendly menu-driven interface.

What Was Learned:

- Practical use of arrays (especially 2D arrays) for data representation.
- Application of functions to create modular and organized code.
- Importance of input validation and boundary checking.
- Effective use of loops and conditional statements in real scenarios.
- Designing a structured and interactive console-based program.

Future Improvement

The project can be improved by allowing different symbols or color representations instead of just numbers, and by increasing or customizing the grid size for more flexible drawing.